

datalab 报告

姓名：（把你的中文名填进来）

学号：2025200706

总分 37/37	bitAnd	bitXor	samesign	logtwo	byteSwap	reverse	logicalShift	leftBitCount	float_i2f	floatScale2	float64_f2i	floatPower2
通过 情况	1/1	1/1	2/2	4/4	4/4	3/3	3/3	4/4	4/4	4/4	3/3	4/4
运算 次数	4/7	7/7	8/12	23/25	10/17	6/30	5/20	29/40	29/30	13/30	16/60	9/30

python3 test.py -v 实测输出：

```
bitAnd      1/1: PASS  using 4 operations  {'~', '|'}
bitXor      1/1: PASS  using 7 operations  {'~', '&'}
samesign    2/2: PASS  using 8 operations  {'^', '>>', '!' } + if
logtwo      4/4: PASS  using 23 operations {'>>', '<<', '|', '>'}
byteSwap    4/4: PASS  using 10 operations {'^', '>>', '&', '<<'}
reverse     3/3: PASS  using 6 operations  {'>>', '+', '&', '<<', '!=', '|'} + for
logicalShift 3/3: PASS  using 5 operations  {'>>', '&', '~', '<<'}
leftBitCount 4/4: PASS  using 29 operations {'>>', '|', '+', '~', '<<', '!=', '|'}
float_i2f    4/4: PASS  using 29 operations {'>>', '|', '+', '==', '~', '<<', '&', '-', '<', '|', '>'} + while
floatScale2  4/4: PASS  using 13 operations {'>>', '+', '==', '&', '<<', '|'}
float64_f2i  3/3: PASS  using 16 operations {'>>', '+', '&', '<<', '~', '-', '<', '|', '>'}
floatPower2  4/4: PASS  using 9 operations  {'+', '<=', '<<', '-', '>='}
Total points: 37
```

解题报告

亮点

- 1. bitXor
- 2. byteSwap

bitXor

```
return ~(x & y) & ~(~x & ~y);
```

核心是利用补集思想，从异或运算的真值表可以看出是不同为1，相同为0。所以既不是都为1、也不是都为 0的情况就是异或运算。

bitAnd

```
return ~(~x | ~y);
```

这是典型的与或运算转换方法，利用结合取反。

samesign

```
if(!x){
    rst = !y;
} else if(!y){
    rst = !x;
} else{
```

```

int signx = x >> 31;
int signy = y >> 31;
rst = !(signx ^ signy);
}

```

这里重点是要利用移位来判断符号（一个全1，一个全0）；除此之外处理0的特殊情况，这里要注意不能用 ‘==’，所以要用 ‘!’ 来处理。

logtwo

```

int r = 0;
int s;
s = ((v >> 16) > 0) << 4; r = r | s; v = v >> s; //这里左移4位是把1转换为16，当做if来用
s = ((v >> 8) > 0) << 3; r = r | s; v = v >> s; //这里v只剩后16位，再次分前8后8
s = ((v >> 4) > 0) << 2; r = r | s; v = v >> s;
s = ((v >> 2) > 0) << 1; r = r | s; v = v >> s;
s = ((v >> 1) > 0); r = r | s;

```

这里的核心思想是二分法判断最高位（因为说了是以2为底的数，不用担心取对后不是整数）。具体步骤是判断前16位，有1就移动到后16位，没有就说明1在后16位，之后就只考虑后16位二分，以此类推。

byteSwap

```

int nB = n << 3;
int mB = m << 3;
int d = ((x >> nB) ^ (x >> mB)) & 0xFF;
return x ^ (d << nB) ^ (d << mB);

```

交换的核心是利用「异或交换法」，而基于这个方法的的操作就是提取需要交换的字节。同时要注意的是利用左移位（3个）把字节序号转为比特序号。

reverse

```

for(int i = 0; i != 32; i++){
    r = (r << 1) | (v & 1);
    v = v >> 1;
}

```

最初打算利用上一题 byteSwap 的思路进行每一对交换，但想到 unsigned 的条件可以直接一位一位取，所以从低位取，填到新的数。

logicalShift

```

x = x >> n;
int d = 0x80000000;
d = ~(d >> n << 1);
x = x & d;

```

核心是利用取反得到0000.....111这样的数，「与」上原值移位能除去算术移位的符号位（1111）三条教训: 1.0x80000000这类 $\geq 2^{31}$ 的十六进制字面量，类型是 unsigned int，对它 >> 是逻辑右移（不补符号位）。想要算术右移必须先赋给 int 变量再移。2.合法表里没有 - 就是不能用，test.py 会直接报 Using illegal operations: { '-' }。3.移位量不能是负数（n-1 在 n=0 时是 UB），所以“少一个1”要用 << 1 挤掉，而不是少移一位。

leftBitCount

```

int v = ~x;
int s = 0;
int r = 0;
return 31 + ~r + 1 + !v; // x = -1时会少一个，用!v来判断，前面是取反加一来代替-

```

这道题是从左边开始数连续1有几个（首0就是0个），利用取反和logtwo取首位的序号（要注意的是logtwo没考虑 $v \leq 0$ 的情况）。所以这里要考虑 $x = -1$ 的情况（ $v < 0$ 没影响），此时这个方法会少算一个。### floatScale2

```
if (exp == 0)    return (uf & 0x80000000u) | (uf << 1); //(uf & 0x80000000u) | 是用来取首位符号位
if (exp == 254)
    if (uf & 0x7FFFFFF) return (uf & 0x80000000u) | 0x7F800000u;
```

这里先想到*2是改变指数位，进一步考虑到两种规格化情况。同时要注意254的情况，这里溢出有规定只能是INF而不是NaN。同时记住固定用法「(uf & 0x80000000u) | 」### floatPower2

```
if (x >= 128)    return 0x7F800000; /* +INF */
if (x <= -150)   return 0;
if (x >= -126)   return (x + 127) << 23; /* 规格化: (x + 127) 挪到 bit23 起 */
return 1 << (149 + x);
```

这道题是将 2^x 转为float表示，那就按照规则来分为规格化和非规格化两种情况。且这道题没有负数，所以不用考虑负无穷和负零。

float_i2f

```
if (!x) return 0;
if (x < 0) { sign = 0x80000000u; ux = ~ux + 1; }
t = ux;
while (t > 1) { t = t >> 1; k = k + 1; } /* k = 最高位1的下标 */
if (k < 24) {
    frac = ux << (23 - k);
} else {
    t = k - 23;
    frac = ux >> t;
    half = 1 << (t - 1);
    rest = ux - (frac << t);
    frac = frac + ((rest > half) | ((rest == half) & (frac & 1)));
}
return sign | ((k + 127 + (frac >> 24)) << 23) | (frac & 0x7FFFFFF);
```

这题的门槛是float只有24位有效精度（23位存储+1位隐含的1），所以先找最高位1的下标k，再以24为界分两种情况： $k < 24$ 时装得下，左移把那个1送到bit23就行，一点不丢； $k \geq 24$ 时要把 $k-23$ 个低位砍掉，就必须舍入。舍入用的是「四舍六入五成双」：把被砍掉的rest和「一半」half比大小，大于就进1，小于就不进，**正好等于一半时看保留部分的末位frac & 1，只有是奇数才进**（进完正好凑成偶数）。最后 $\text{frac} \gg 24$ 是处理「舍入进位把24位撑成25位」的情况，多出来的那个1直接加到阶码上，省掉一个if。

float64_f2i

```
unsigned sign = uf2 >> 31;
unsigned exp  = (uf2 >> 20) & 0x7FF;
unsigned val  = 0x80000000u | ((uf2 & 0xFFFF) << 11) | (uf1 >> 21);
if (exp < 1023) return 0;
if (exp > 1054) return 0x80000000;
val = val >> (1054 - exp);
if (sign) {
    if (val > 0x80000000u) return 0x80000000;
    return ~val + 1;
}
if (val > 0x7FFFFFFF) return 0x80000000;
return val;
```

double的52位尾数装不进一个32位字，所以只取最高的31位、和隐含的1拼成val。这样 $\text{val} = \text{真实尾数} \times 2^{31}$ （因为那个隐含的1被放在了bit31上）。要还原成整数就得抵消两件事：阶码里的偏移1023，和上面放大的 2^{31} ，合起来就是右移 $1054 - \text{exp}$ 位， $1054 = 1023 + 31$ 。读字段时用 $(\text{uf2} \gg 20) \& 0x7FF$ ：阶码下面压着20位尾数所以移20，

0x7FF 是 11 个 1 正好对应 double 的 11 位阶码。边界: $\text{exp} < 1023$ 说明绝对值不到 1, 向零舍入就是 0; $\text{exp} > 1054$ 超出 int 范围返回 0x80000000; $\text{exp} == 1054$ 还要再看一眼 val 有没有越过 $2^{31}-1$ 。

反馈/收获/感悟/总结

参考的重要资料
